

Guía de Eficiencia en Eclipse para Java

Atajos de teclado, opciones útiles y buenas prácticas

1. Introducción

Eclipse IDE es la herramienta elegida en la ETSIST para el desarrollo de aplicaciones Java. Vas a tener que utilizarla, no solo en Programación II, sino en asignaturas posteriores, como Programación Avanzada de Aplicaciones o Software de Comunicaciones. Si en tu futura actividad profesional tienes que programar en Java, Eclipse es una buena opción para desarrollar los programas. Conocer bien sus funcionalidades puede marcar una gran diferencia en tu productividad como programador.

Este documento recoge los atajos de teclado más útiles, así como opciones de configuración y características del entorno que te permitirán trabajar de forma más ágil y eficiente durante las prácticas de la asignatura y en tu futuro profesional.

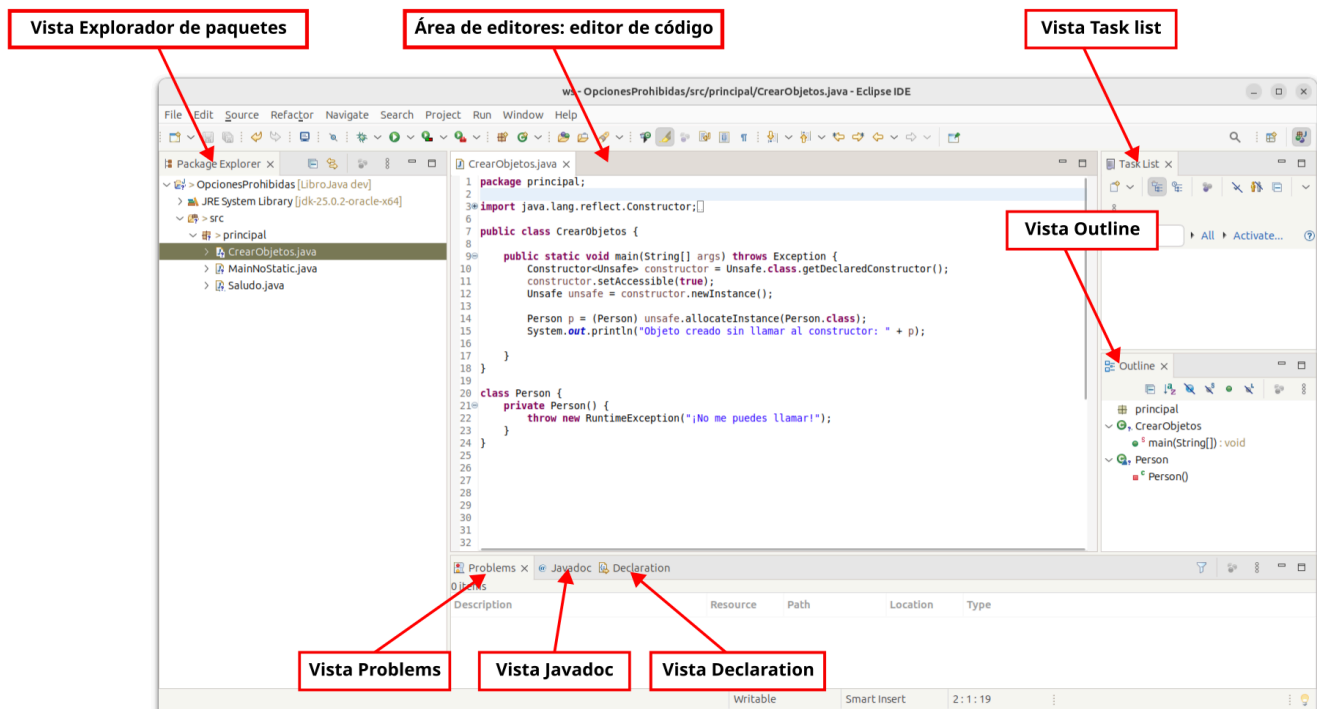
 **Consejo:** en general, todos los atajos de teclado están asociados a una opción que se puede seleccionar en algún menú o mediante alguna acción de ratón, pero es mucho más eficiente aprender a utilizar atajos de teclado. Dedica unos días a practicar estos atajos conscientemente. Al principio puede parecer lento, pero en poco tiempo serán automáticos y ahorrarás mucho tiempo en cada sesión de trabajo.

 **Nota:** en el Moodle de Programación II, en la sección "**Recursos docentes por profesor -> Santiago Higuera**", puedes encontrar la versión pdf de este artículo, que te servirá para tenerla a mano y poder consultarla cuando la necesites.

2. Gestión de vistas y perspectivas

Eclipse organiza su interfaz (*workbench*, *banco de trabajo*) en *perspectivas*. Cada perspectiva es un conjunto de *vistas* y cada *vista* permite visualizar determinados contenidos. Por ejemplo, el explorador de paquetes que se muestra a la izquierda de la ventana de Eclipse es una vista. Además, Eclipse muestra en la parte central el *área de editores*, que es donde visualizaremos el contenido de los archivos.

Eclipse tiene una serie de perspectivas predefinidas. La **perspectiva Java**, que es la que usaremos normalmente al hacer los programas, muestra la vista del *explorador de paquetes* a la izquierda, el *editor de código* en el centro, las vistas *outline* y *Task list* a la derecha y las vistas de *problemas*, *javadoc* y otras en la parte inferior. La figura siguiente muestra la perspectiva Java original, con indicación de las vistas que utiliza:



Hay otras perspectivas. Cuando depuremos programas, por ejemplo, se puede activar la *perspectiva de depuración*, que muestra otra serie de vistas ordenadas de cierta forma.

Desde el menú "*Window -> Show View*", podemos abrir nuevas vistas. Cada vista la podemos situar en el panel que elijamos, pichando en su pestaña y arrastrándola al lugar deseado. También podemos separarlas de la ventana principal, pichando con el botón derecho en su pestaña y seleccionando *detach*. Para volver a llevarla a la ventana principal, podemos pinchar en la pestaña y arrastrarla.

Podemos seleccionar la perspectiva que queremos activar en la opción de menú "*Window -> Perspective -> Open Perspective*".

Además de las perspectivas que trae el programa definidas por defecto, podemos crear nuestras propias perspectivas para utilizar en nuestros proyectos. El menú "*Window -> Perspective*" ofrece opciones para ello.

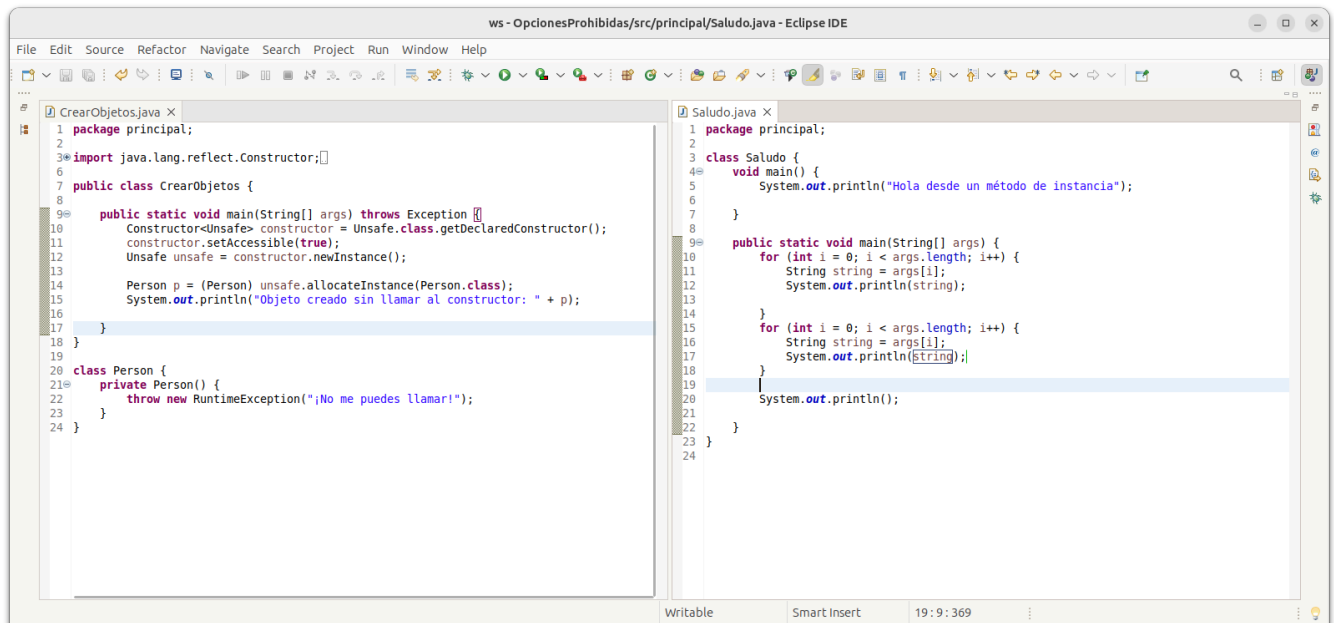
También podemos restaurar la disposición original de una perspectiva seleccionando "*Window -> Perspective -> Reset Perspective*".

2.1 Vistas recomendadas

- **Package Explorer:** navegación por la estructura del proyecto. Actívalo siempre.
- **Project Explorer:** es similar a la anterior, pero permite ver todas las carpetas del proyecto, por ejemplo la carpeta *bin* con los archivos compilados. En un artículo anterior en este mismo blog expliqué cómo hacer para ver la carpeta *bin*.
- **Outline:** muestra la estructura del archivo actual (métodos, atributos). Muy útil en clases largas.
- **Console:** salida del programa y mensajes de error. Imprescindible en todo momento.
- **Problems:** lista de errores y advertencias. Haz clic en cada problema para ir directo a la línea.
- **Javadoc:** muestra la documentación de la clase o método bajo el cursor automáticamente.
- **Call Hierarchy** (**Ctrl + Alt + H**): muestra quién llama a un método y a quién llama él.
- **Type Hierarchy** (**F4**): muestra la jerarquía de herencia de una clase.

2.2 Gestión de ventanas en el editor

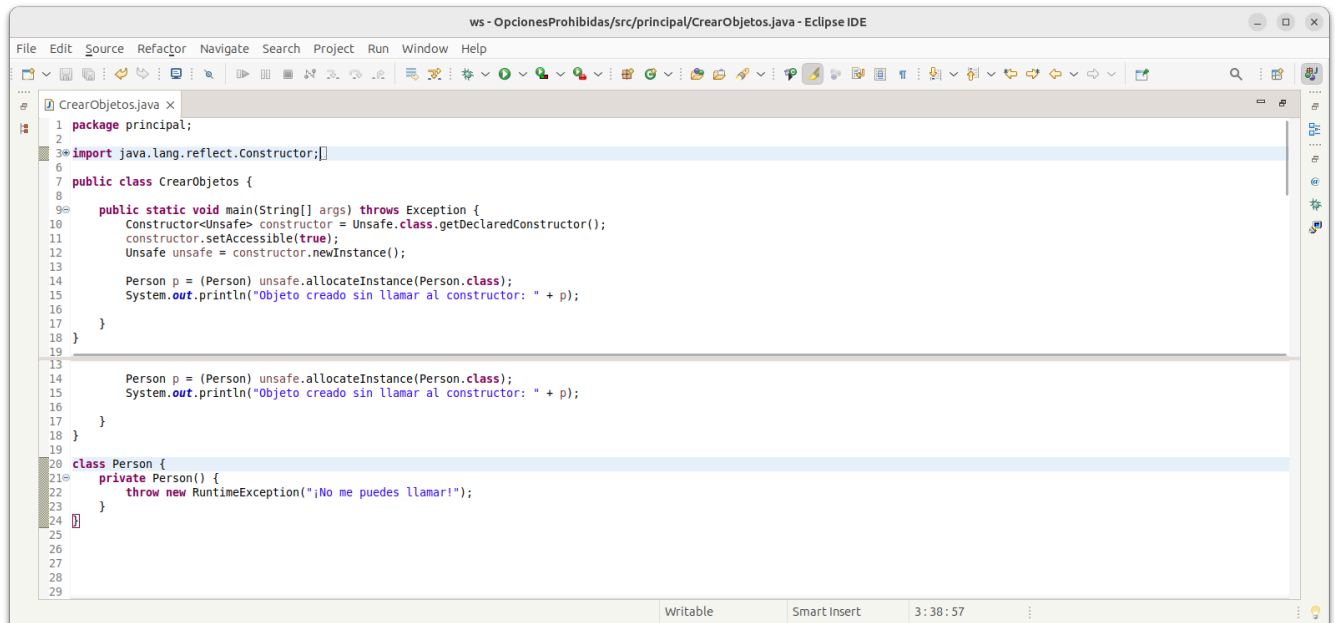
Sabemos que se pueden tener varias pestañas abiertas en el editor, cada una correspondiente a una clase o fichero diferente. Pero también podemos partir la ventana del editor en horizontal o en vertical, para visualizar simultáneamente varios ficheros, como se ha hecho en la imagen siguiente:



Seguramente, la forma más cómoda de hacerlo sea arrastrando con el ratón una de las pestañas hacia el borde de la ventana. En la imagen anterior se ha hecho además otro ajuste: pulsando **Ctrl + M** se maximiza la ventana del editor, ocultando otros paneles, como el panel del explorador de paquetes u otros que hubiera abiertos. Para restituir la vista de dichos paneles solo hay que volver a pulsar **Ctrl + M**.

Podemos mostrar en una de las subventanas cualquier otra vista que tengamos abierta, por ejemplo el terminal u otra. El procedimiento es el mismo: pinchar en la pestaña de la vista que queremos mover y arrastrarla hasta el punto donde queremos visualizarla.

También es posible mostrar en dos paneles el mismo fichero, situado en diferentes posiciones de dicción (*split*). Esto puede facilitar la edición del código en ficheros de gran tamaño. Para mostrar un mismo fichero en dos paneles en horizontal hay que teclear **Ctrl + AltGr + {**. Una nueva pulsación del atajo de teclado indicado, desactivará la opción. Para colocar los dos paneles uno encima del otro hay que pulsar **Ctrl + Shift + -**.



2.3 Atajos de vistas y perspectivas

Atajo	Descripción
Ctrl + M	Maximizar o restaurar el editor o vista actual
Ctrl + F8	Cambiar entre perspectivas (Java, Debug, Git...)
F12	Volver el foco al editor de código
Ctrl + AltGr + {	Split horizontal
Ctrl + Shift + -	Split vertical

3. Atajos para gestión de ficheros

Atajo	Descripción
Ctrl + N	New: abre el diálogo File ->New
Ctrl + S	Guardar

4. Atajos Esenciales de Edición de Código

Estos son los atajos que usarás a diario al escribir código Java. Merece la pena aprenderlos desde el primer día.

4.1 Escritura y formato

Atajo	Descripción
Ctrl + Espacio	Autocompletado de código: completa nombres de clases, métodos, variables y genera plantillas (sysout , for , if , try ...)

Atajo	Descripción
<code>Ctrl + Shift + F</code>	Formatea automáticamente todo el archivo según las convenciones de Java
<code>Ctrl + D</code>	Elimina la línea actual completa sin necesidad de seleccionarla
<code>Alt + ↑ / Alt + ↓</code>	Mueve la línea actual (o selección) hacia arriba o hacia abajo
<code>Ctrl + Alt + ↑ / ↓</code>	Duplica la línea actual hacia arriba o hacia abajo
<code>Ctrl + Shift + 7</code>	Comenta o descomenta la línea actual con <code>//</code> (en teclado español, <code>/</code> equivale a <code>Shift + 7</code>)
<code>Ctrl + Shift + 7</code> (bloque)	En teclado español, este mismo atajo sobre una selección comenta las líneas individualmente con <code>//</code> . Para comentario de bloque <code>/* ... */</code> no hay atajo directo: usad el menú <i>Source → Toggle Block Comment</i>
<code>Tab / Shift + Tab</code>	Aumenta o reduce la indentación del bloque seleccionado
<code>Ctrl + Z / Ctrl + Y</code>	Deshacer / Rehacer última acción
<code>Ctrl + C</code>	Copy
<code>Ctrl + V</code>	Paste
<code>Ctrl + X</code>	Cut

💡 **Consejo:** `Ctrl + Espacio` es el atajo más poderoso de Eclipse. Úsalo constantemente: no solo completa palabras, también genera estructuras completas como bucles for-each, bloques try-catch, el método `main`, etc.

Por ejemplo, para escribir `System.out.println()`, solo tendrás que teclear `sysout` seguido de `Ctrl + espacio`. Más adelante, en el apartado sobre *Generación automática de código* ampliamos información sobre este tema.

4.2 Navegación dentro del código

Atajo	Descripción
<code>Ctrl + Clic / F3</code>	Ir a la definición de la clase, método o variable bajo el cursor
<code>Alt + ← / Alt + →</code>	Retroceder / avanzar en el historial de navegación
<code>Ctrl + L</code>	Ir a una línea concreta del archivo por número
<code>Ctrl + Q</code>	Volver a la última posición editada

Atajo	Descripción
<code>Ctrl + E</code>	Mostrar lista de editores abiertos para cambiar rápidamente entre archivos
<code>Ctrl + F6</code>	Alternar entre los editores abiertos (como Alt+Tab del sistema)
<code>Ctrl + Shift + G</code>	Buscar todas las referencias a un método o variable en el proyecto

4.3 Selección de texto

Atajo	Descripción
<code>Shift + Alt + ↑</code>	Ampliar la selección al elemento sintáctico superior (seleccionar bloques completos)
<code>Shift + Alt + ↓</code>	Reducir la selección al elemento sintáctico inferior
<code>Ctrl + A</code>	Seleccionar todo el contenido del archivo
<code>Alt + Shift + A</code>	Activar modo de selección por columnas (selección rectangular)

5. Búsqueda y Reemplazo

Atajo	Descripción
<code>Ctrl + F</code>	Abrir el panel de búsqueda en el archivo actual
<code>Ctrl + H</code>	Búsqueda avanzada en todo el proyecto (por archivos, expresiones regulares...)
<code>Ctrl + K</code>	Buscar la siguiente ocurrencia del texto seleccionado
<code>Ctrl + Shift + K</code>	Buscar la ocurrencia anterior del texto seleccionado
<code>Ctrl + J</code>	Búsqueda incremental: encuentra resultados en tiempo real mientras escribes
<code>Ctrl + Shift + R</code>	Abrir un recurso (archivo) del proyecto buscando por nombre
<code>Ctrl + Shift + T</code>	Buscar y abrir un tipo (clase, interfaz, enum) de todo el workspace
<code>Ctrl + O</code>	Mostrar el esquema del archivo actual: métodos, atributos, constructores

 **Consejo:** `Ctrl + Shift + T` y `Ctrl + Shift + R` son imprescindibles en proyectos grandes. Permiten localizar cualquier clase o archivo instantáneamente sin navegar por el Package Explorer.

6. Refactorización de Código

Eclipse ofrece potentes herramientas de refactorización que modifican el código de forma segura, actualizando automáticamente todas las referencias en el proyecto.



Nota: se denomina **refactorizar** a modificar el código de un programa sin afectar a su funcionamiento. Por ejemplo, cambiar el nombre a una variable o método es una refactorización frecuente.

Atajo	Descripción
Alt + Shift + R	Renombrar una clase, método, variable o paquete actualizando todas las referencias
Alt + Shift + M	Extraer el código seleccionado a un nuevo método
Alt + Shift + L	Extraer la expresión seleccionada a una variable local
Alt + Shift + I	Hacer inline: sustituir el uso de una variable por su valor directamente
Alt + Shift + C	Cambiar la firma de un método (parámetros, nombre, visibilidad)
Alt + Shift + T	Abrir el menú de refactorización completo sobre el elemento seleccionado
Alt + Shift + S	Abrir el menú Source: generar getters/setters, constructores, toString...



Consejo: Usa siempre Alt + Shift + R para renombrar, nunca lo hagas manualmente con buscar y reemplazar. Eclipse actualiza todas las referencias del proyecto de forma segura y no os perderéis ninguna ocurrencia.

7. Generación Automática de Código

Eclipse puede generar por ti mucho código repetitivo. Accede al menú **Source** (Alt + Shift + S) para ver todas las opciones disponibles:

Atajo	Descripción
Alt + Shift + S, R	Generar métodos getter y setter para los atributos de la clase
Alt + Shift + S, O	Generar constructores usando campos de la clase
Alt + Shift + S, H	Generar los métodos <code>hashCode()</code> y <code>equals()</code>
Alt + Shift + S, S	Generar el método <code>toString()</code>

Atajo	Descripción
<code>Alt + Shift + S</code> , <code>V</code>	Implementar métodos abstractos no implementados de interfaces o clases abstractas
<code>Alt + Shift + S</code> , <code>U</code>	Generar métodos de superclase no implementados
<code>Ctrl + 1</code>	Correcciones rápidas: resolver imports, crear métodos, capturar excepciones, etc.

 **Consejo:** `Ctrl + 1` (Quick Fix) es uno de los atajos más valiosos. Si Eclipse marca un error en rojo, pulsad `Ctrl + 1` y os ofrecerá soluciones automáticas: añadir imports, crear métodos que faltan, encapsular en try-catch...

7.1 Plantillas de código (Templates)

Eclipse tiene plantillas de código que se activan escribiendo la abreviatura y pulsando `Ctrl + Espacio`. Por ejemplo, si tecleas `main`, seguido de `Ctrl + espacio`, se escribirá el método `main()` completo.

Abreviatura	Genera
<code>sysout</code>	<code>System.out.println();</code>
<code>syserr</code>	<code>System.err.println();</code>
<code>for</code>	Bucle for clásico con índice
<code>foreach</code>	Bucle for-each sobre una colección
<code>try</code>	Bloque try-catch-finally
<code>main</code>	<code>public static void main(String[] args)</code>
<code>ifelse</code>	Estructura if-else completa

Puedes crear tus propias plantillas en: `Window → Preferences → Java → Editor → Templates`.

8. Depuración (Debug)

El depurador de Eclipse es una herramienta fundamental para entender el comportamiento de tus programas y localizar errores lógicos de forma eficiente.

Atajo	Descripción
<code>F11</code>	Ejecutar en modo Debug (lanza la última configuración usada)
<code>Ctrl + Shift + B</code>	Añadir o quitar un Breakpoint en la línea actual
<code>F8</code>	Continue: continuar la ejecución hasta el siguiente breakpoint
<code>F6</code>	Step Over: ejecutar la línea actual sin entrar en el método llamado

Atajo	Descripción
F5	Step Into: entrar dentro del método llamado en la línea actual
F7	Step Return: salir del método actual y volver al llamador
Ctrl + Shift + I	Inspect: evaluar una expresión o variable en el contexto actual
Ctrl + F2	Terminar la sesión de depuración

 **Consejo:** Para depurar eficientemente: pon breakpoints estratégicos al inicio de métodos sospechosos, usa la vista **Variables** para inspeccionar el estado, y añade **Watch Expressions** para evaluar expresiones complejas en tiempo real.

9. Opciones de Configuración Útiles

9.1 Compilación automática

Window → Preferences → General → Workspace: activad **Build automatically** para que Eclipse compile y detecte errores al instante.

9.2 Acciones al guardar (Save Actions)

Window → Preferences → Java → Editor → Save Actions: configurad que al guardar (Ctrl + S) Eclipse formatee el código y organice los imports automáticamente.

 **Consejo:** Activar el formateo automático al guardar es una de las mejores decisiones que puedes tomar. Tu código siempre estará bien indentado sin ningún esfuerzo extra.

9.3 Otros ajustes recomendados

- **Números de línea:** clic derecho en el margen izquierdo del editor → *Show Line Numbers*.
- **Tema oscuro:** Window → Preferences → General → Appearance → Theme 'Dark'. Reduce la fatiga visual en sesiones largas.
- **Organizar imports:** Ctrl + Shift + O completa imports pendientes y elimina imports innecesarios del archivo actual.

10. Abrir Archivos en el Explorador del Sistema y el Terminal

En ocasiones necesitarás acceder a los archivos del proyecto desde fuera de Eclipse: para compartir ficheros, abrirlos con otra aplicación o lanzar un terminal en esa ubicación. Eclipse facilita esto directamente desde el Package Explorer.

10.1 Abrir la ubicación en el explorador de archivos del sistema

Haz clic derecho sobre el fichero o carpeta en el **Package Explorer** y selecciona:

Show In → System Explorer

Esto abrirá el explorador de archivos del sistema operativo (**Explorador de Windows**, **Finder** en macOS o el gestor de archivos en Linux) con el fichero o carpeta ya seleccionado en su ubicación real en el disco.

 **Consejo:** Esta opción es especialmente útil para localizar rápidamente dónde está guardado tu proyecto en el disco, sin tener que buscar manualmente la carpeta del workspace de Eclipse.

10.2 Abrir un terminal externo desde esa ubicación

Una vez en el explorador de archivos del sistema operativo, puedes abrir un terminal situado directamente en esa carpeta sin necesidad de navegar manualmente hasta ella:

- **Windows:** haz clic derecho sobre la carpeta (o sobre el fondo de la ventana si ya estás dentro) y selecciona *Abrir en Terminal*.
- **macOS:** clic derecho sobre la carpeta → *Servicios* → *Nuevo Terminal en la Carpeta* (puede requerir activarlo en *Preferencias del Sistema* → *Teclado* → *Servicios*).
- **Linux:** clic derecho sobre la carpeta → *Abrir en Terminal* (el nombre exacto depende del gestor de archivos).

De esta forma puedes ejecutar comandos directamente en el directorio del proyecto, por ejemplo para compilar con `javac` o ejecutar con `java`.

11. Personalización de Atajos de Teclado

Eclipse permite consultar y modificar todos los atajos desde un único panel, lo que resulta especialmente útil para **resolver conflictos** o **asignar atajos a comandos que no los tienen** (como ocurre con *Toggle Block Comment* en teclado español).

11.1 Cómo acceder al panel de atajos

Window → Preferences → General → Keys

O directamente pulsando `Ctrl + Shift + L` dos veces seguidas.

11.2 Consultar combinaciones en uso

En el panel de Keys encontrarás una tabla con todos los comandos del IDE. Puedes filtrarla de dos formas:

- **Por nombre de comando:** escribe parte del nombre en el campo *Filter* para ver si ya tiene atajo asignado y cuál es.
- **Por combinación de teclas:** haz clic en el campo *Binding* y pulsa la combinación que quieres comprobar. Eclipse mostrará si ya está asignada a algún comando.

Ten en cuenta que un mismo atajo puede estar libre en un **contexto** (*Context*) aunque esté siendo utilizado en otro. Por ejemplo, un atajo de teclado puede funcionar cuando estás en el editor de código y no funcionar en otras vistas.

11.3 Asignar o modificar un atajo

1. Busca el comando deseado en el campo *Filter* (por ejemplo, `Toggle Block Comment`).
2. Selecciona el comando en la tabla.
3. Haz clic en el campo **Binding** y pulsa la combinación de teclas que quieras asignar.
4. Comprueba que el campo **Conflicts** no muestre ningún conflicto.
5. Pulsa **Apply and Close**.

 **Consejo:** Si al pulsar la combinación Eclipse detecta un conflicto, aparecerá resaltado en la sección *Conflicts*. En ese caso puedes eliminar el atajo del comando conflictivo (si no lo usas) o elegir otra combinación libre.

11.4 Ejemplo: asignar un atajo a Toggle Block Comment en teclado español

El comando *Toggle Block Comment* (comentario de bloque `/* ... */`) no tiene atajo funcional en teclado español. Para asignárselo:

1. Abrid `Window → Preferences → General → Keys`.
2. En *Filter*, escribe `Toggle Block Comment`.
3. Selecciona el comando y haz clic en *Binding*.
4. Pulsa la combinación que quieras, por ejemplo `Ctrl + Shift + 8` (comprueba que no haya conflictos).
5. Pulsa **Apply and Close**.

Atajo	Descripción
<code>Ctrl + Shift + L</code>	Mostrar la lista completa de todos los atajos disponibles
<code>Ctrl + Shift + L</code> (x2)	Abrir las preferencias de atajos de teclado para personalizarlos

12. Productividad en el Editor

12.1 Múltiples cursores y edición en bloque

Eclipse permite editar varias líneas simultáneamente mediante la **selección por columnas**, también llamada selección rectangular. Es muy útil para modificar código repetitivo en varias líneas a la vez.

Para activarla, pulsa `Alt + Shift + A` y luego selecciona con el ratón (o con las teclas de dirección) el bloque de líneas que quieres editar. Todo lo que escribas se aplicará en todas las líneas seleccionadas al mismo tiempo.

 **Consejo:** Es especialmente útil para añadir o eliminar un prefijo o sufijo en varias líneas a la vez, como comentar un bloque de declaraciones o alinear asignaciones.

12.2 Comparar dos archivos entre sí

Eclipse permite comparar el contenido de dos archivos directamente desde el Package Explorer, sin necesidad de herramientas externas. Selecciona los dos archivos manteniendo `Ctrl` pulsado, haz clic derecho y elige:

`Compare With → Each Other`

Se abrirá una vista de diferencias que muestra lado a lado el contenido de ambos archivos, resaltando las líneas que difieren. Es muy útil para comparar versiones de una práctica o revisar cambios entre dos implementaciones.

13. Comprensión del Código

13.1 Call Hierarchy y Type Hierarchy

Estas dos vistas ayudan a entender la estructura y el flujo de un programa, especialmente cuando se trabaja con herencia y polimorfismo.

Call Hierarchy (`Ctrl + Alt + H` sobre un método): muestra un árbol con todos los métodos que llaman al método seleccionado (*Caller Hierarchy*) y todos los métodos a los que él llama (*Callee Hierarchy*). Es muy útil para entender el impacto de un cambio o seguir el flujo de ejecución.

Type Hierarchy (`F4` sobre una clase o interfaz): muestra la jerarquía completa de herencia, tanto hacia arriba (superclases e interfaces implementadas) como hacia abajo (subclases). Imprescindible cuando trabajéis con herencia para tener una visión global de la jerarquía de clases.

13.2 Javadoc emergente al pasar el ratón

Simplemente dejando el cursor del ratón sobre el nombre de una clase, método o variable durante un momento, Eclipse muestra automáticamente una ventana emergente con su documentación Javadoc: descripción, parámetros, valor de retorno y excepciones.

No es necesario ningún atajo: basta con **sobrevolar** con el ratón. Si quieres verlo de forma fija en una vista permanente, activa la vista **Javadoc** desde `Window → Show View → Javadoc`, que se actualiza automáticamente según el elemento bajo el cursor.

 **Consejo:** Aprovecha esta función constantemente al usar las clases de la API de Java (listas, colecciones, String, etc.). Es mucho más rápido que buscar en la documentación oficial.

14. Gestión del Proyecto

14.1 Exportar el proyecto como archivo `.jar`

Cuando quieras entregar una práctica como ejecutable, o reutilizar tu código en otro proyecto, puedes empaquetarlo como un archivo `.jar`:

1. Clic derecho sobre el proyecto en el Package Explorer → `Export...`
2. Selecciona `Java → Runnable JAR file` (si quieres un JAR ejecutable) o `Java → JAR file` (si es una librería).

3. Elige la clase que contiene el método `main` como punto de entrada (solo para JAR ejecutable).
4. Indica la ruta de destino y pulsa **Finish**.

 **Consejo:** Un *Runnable JAR* puede ejecutarse directamente con `java -jar archivo.jar` desde el terminal, sin necesidad de tener Eclipse instalado.

14.2 Incorporar un archivo `.jar` externo al classpath del proyecto

Cuando necesites usar una librería externa (por ejemplo, una proporcionada por el profesor o descargada de Internet), debes añadir su `.jar` al classpath del proyecto para que Eclipse la reconozca:

1. Copia el archivo `.jar` en una carpeta dentro del proyecto (por convenio, se suele llamar `lib/`).
2. Clic derecho sobre el archivo `.jar` en el Package Explorer → `Build Path → Add to Build Path`.

A partir de ese momento Eclipse reconocerá las clases de esa librería y ofrecerá autocompletado y Javadoc para ellas (si el `.jar` incluye documentación).

 **Consejo:** Si el `.jar` no aparece en el Package Explorer, comprobad que está dentro de la carpeta del proyecto. Si lo tenéis en otra ubicación del disco, usad `Build Path → Configure Build Path → Libraries → Add External JARs...` en su lugar.

14.3 Importar y exportar preferencias

Si trabajas en varios equipos (por ejemplo, en casa y en el laboratorio), puedes exportar toda tu configuración de Eclipse —atajos personalizados, plantillas, formato de código— y recuperarla en otro equipo.

- **Exportar:** `File → Export → General → Preferences`. Guardad el archivo `.epf` resultante.
- **Importar:** `File → Import → General → Preferences`. Seleccionad el archivo `.epf`.

 **Consejo:** Guardad tu archivo `.epf` en la nube o en el mismo repositorio Git del proyecto para tenerlo siempre disponible.

14.4 Working Sets

Cuando tengas varios proyectos abiertos a la vez en el workspace, el Package Explorer puede volverse difícil de manejar. Los **Working Sets** permiten agrupar proyectos relacionados y mostrar solo los que os interesan en cada momento.

Para crearlos: haz clic en el menú desplegable del Package Explorer (icono de flecha en la esquina superior derecha) → `Select Working Set...` → `New...`.

15. Calidad del Código

15.1 Warnings y nivel de severidad

Eclipse no solo detecta errores de compilación (marcados en rojo), sino también **advertencias** (*warnings*, marcadas en amarillo) que señalan código potencialmente problemático: variables declaradas pero no usadas, conversiones implícitas entre tipos, comparaciones innecesarias, etc.

Es recomendable prestarles atención aunque no impidan compilar. Puedes configurar qué situaciones generan error, warning o se ignoran en:

Window → Preferences → Java → Compiler → Errors/Warnings

 **Consejo:** Un código sin warnings es señal de mayor calidad y rigor. Acostúmbrate a mantener la vista *Problems* limpia, no solo de errores sino también de advertencias.

15.2 Task Tags: TODO, FIXME y XXX

Eclipse reconoce automáticamente ciertos comentarios especiales en el código y los muestra como tareas pendientes en la vista **Tasks** (Window → Show View → Tasks):

Etiqueta	Uso habitual
// TODO	Funcionalidad pendiente de implementar
// FIXME	Código que funciona pero necesita ser corregido
// XXX	Código problemático o cuestionable que requiere atención

Por ejemplo:

```
// TODO implementar el manejo de excepciones
// FIXME este cálculo falla con valores negativos
```

Estas etiquetas aparecerán listadas en la vista Tasks, lo que permite llevar un registro de las partes del código que aún necesitan trabajo sin perder el hilo.

 **Consejo:** Usa `// TODO` mientras desarrollas para marcar partes incompletas, y eliminalos antes de entregar la práctica. Un TODO en el código entregado indica que sabes que algo falta.

16. Tabla Resumen — Cheat Sheet

Los 20 atajos que debes conocer sí o sí:

Atajo	Descripción
Ctrl + Espacio	Autocompletar código
Ctrl + 1	Corrección rápida (Quick Fix)
Ctrl + Shift + F	Formatear código
Ctrl + Shift + O	Organizar imports
Ctrl + D	Eliminar línea actual
Ctrl + S	Guardar (Save)
Ctrl + /	Comentar / descomentar línea
Ctrl + Z / Y	Deshacer / Rehacer
F3 / Ctrl + Clic	Ir a definición

Atajo	Descripción
Alt + ← / →	Navegar historial de edición
Ctrl + Shift + T	Buscar tipo (clase)
Ctrl + Shift + R	Buscar recurso (archivo)
Ctrl + O	Esquema del archivo actual
Alt + Shift + R	Renombrar (refactoring)
Alt + Shift + M	Extraer método
Alt + Shift + S	Menú Source (generar código)
Ctrl + Shift + B	Añadir / quitar breakpoint
F5 / F6 / F7 / F8	Debug: Step Into / Over / Return / Continue
Ctrl + M	Maximizar editor
Ctrl + Shift + L	Ver todos los atajos disponibles

¡La práctica hace al maestro! Cuanto más uses estos atajos, más naturales te resultarán.